

Documento Técnico — Simulador de Escalonamento de Processos

Trabalho 01 da disciplina CK0234 - Sistemas Operacionais (UFC)

Trabalho 01 da disciplina CK0234 — Sistemas Operacionais (UFC).

1. Visão geral

O simulador lê um conjunto de processos da entrada padrão e um arquivo com quantum e aging, executa os sete algoritmos sobre a mesma entrada e apresenta, para cada um:

- turnaround médio (tt);
- espera média (tw);
- número de trocas de contexto;
- diagrama de tempo, com uma linha por segundo.

Há duas formas de uso sobre o mesmo núcleo:

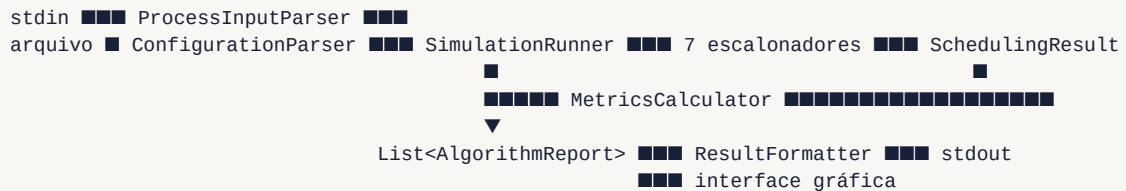
Forma	Ponto de entrada	Execução
Terminal	br.ufc.so.escalonamento.Main	./gradlew -q run --args="config.txt" < processos.txt
Interface gráfica	br.ufc.so.escalonamento.gui.GuiMain	./gradlew runGui

Tecnologia: Java 21, Gradle Wrapper, JUnit 5 e Swing, sem dependências de execução externas ao JDK.

2. Organização em pacotes

```
br.ufc.so.escalonamento
  ■■■ Main, SchedulerApplication  aplicação de terminal
  ■■■ domain                    ProcessControlBlock, ProcessState
  ■■■ input                    ProcessInputParser, ConfigurationParser, SchedulerConfiguration,
  ■                             InputValidationException, ByteOrderMark
  ■■■ scheduler                FcfsScheduler, SjfScheduler, SrtfScheduler,
  ■                             NonPreemptivePriorityScheduler, PreemptivePriorityScheduler,
  ■                             RoundRobinScheduler, PriorityRoundRobinScheduler,
  ■                             SchedulingResult, SchedulingSupport
  ■■■ metrics                  MetricsCalculator, SchedulingMetrics
  ■■■ simulation                SimulationRunner, AlgorithmReport
  ■■■ output                    ResultFormatter
  ■■■ gui                      GuiMain, SchedulerWindow, AlgorithmPanel, ComparisonPanel,
  ■                             GanttChartPanel, TimelinePresentation, modelos de tabela
```

Fluxo de uma execução pelo terminal:



Dependências entre pacotes:

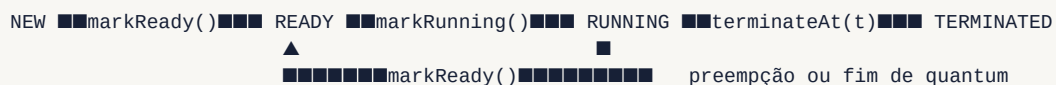
- scheduler não conhece entrada, saída nem interface;
- simulation apenas orquestra;
- output e gui apenas apresentam resultados.

3. Estrutura de controle do processo (PCB)

Cada processo é representado por `domain.ProcessControlBlock`, que reúne sua identificação, seus tempos, suas prioridades e seu estado.

Campo	Tipo	Mutável	Significado
id	int	não	Identificador P1, P2... na ordem das linhas da entrada.
arrivalTime	int	não	Instante de criação.
duration	int	não	Tempo total de CPU necessário, em segundos.
staticPriority	int	não	Prioridade estática; maior número = maior prioridade (notas de aula).
remainingTime	int	sim	Tempo de CPU que ainda falta; começa igual a <code>duration</code> .
dynamicPriority	int	sim	Prioridade dinâmica; começa igual à estática e recebe <code>aging</code> .
state	ProcessState	sim	NEW, READY, RUNNING ou TERMINATED.
completionTime	Integer	sim	Instante de conclusão; null até o término.

Transições permitidas, com validação do estado de origem:



Operações auxiliares:

- `executeOneSecond()`: decrementa o tempo restante; só é permitida em `RUNNING`.
- `applyAging(taxa)`: incrementa a prioridade dinâmica; só é permitida em `READY`.
- `resetDynamicPriority()`: restaura a prioridade estática; só é permitida em `RUNNING`.
- `freshCopy()`: cria um PCB novo, em `NEW`, com os mesmos atributos de identidade.

Motivações:

- Transições explícitas e validadas. Um erro de algoritmo, como executar um processo que não está em `RUNNING` ou envelhecer o processo em execução, lança exceção e aparece nos testes, em vez de produzir um resultado errado em silêncio.
- Não há estado de bloqueio. O enunciado não modela entrada e saída.

- Cópias por algoritmo. Cada escalonador chama `freshCopy()` sobre a entrada. Assim, os sete algoritmos recebem os mesmos processos sem compartilhar estado mutável, e a lista original nunca é alterada.

4. Estruturas de dados por algoritmo

Algoritmo	Classe	Fila de prontos	Processo em execução	Seleção
FCFS	FcfsScheduler	ArrayList	variável local no laço interno	menor criação
SJF	SjfScheduler	ArrayList	variável local no laço interno	menor duração
SRTF	SrtfsScheduler	ArrayList	variável <code>runningProcess</code> , fora da lista	menor tempo restante, reavaliado a cada segundo
Prioridade sem preempção	NonPreemptivePriorityScheduler	ArrayList	variável local no laço interno	maior prioridade estática
Prioridade com preempção	PreemptivePriorityScheduler	ArrayList	variável <code>runningProcess</code> , fora da lista	maior prioridade estática, reavaliada a cada segundo
Round-Robin	RoundRobinScheduler	ArrayDeque (FIFO)	variável local no laço do quantum	cabeça da fila
Round-Robin com prioridade e aging	PriorityRoundRobinScheduler	ArrayList	variável local no laço do quantum	maior prioridade dinâmica, a cada quantum

Justificativas:

- ArrayList como fila de prontos. Nos algoritmos com critério de seleção, a escolha percorre todos os prontos em três passagens explícitas: melhor critério, menor tempo restante e empatados. Uma fila de prioridade (PriorityQueue) esconderia o critério em um comparador e não resolveria o desempate aleatório nem a preferência por quem já ocupa a CPU. Com as entradas do trabalho, o custo linear é irrelevante.
- ArrayDeque no Round-Robin. Na versão sem prioridade, a ordem de chegada à fila é a própria regra de seleção: `pollFirst()` escolhe e `addLast()` recoloca o processo preemptado.
- Processo em execução fora da lista nos preemptivos. Separar `runningProcess` dos prontos torna visíveis a comparação que decide a preempção e o primeiro critério de desempate do enunciado (manter quem já ocupa a CPU).
- Linha do tempo como `List<Integer>`. Cada posição `t` representa o intervalo $[t, t+1)$. O valor é o identificador do processo, ou `SchedulingResult.IDLE (0)` quando a CPU está ociosa.

5. Funcionamento dos algoritmos

Todos seguem o mesmo esqueleto, escrito por extenso em cada classe para que a regra de cada um fique visível:

```

enquanto houver processo não terminado:
    admitir processos cuja criação chegou (NEW → READY)
    se não houver pronto: registrar OCIOSO e avançar 1 s
    [regra específica de seleção, execução e preempção]

```

5.1 FCFS, SJF e prioridade sem preempção

```

selecionado ← melhor pronto segundo o critério
READY → RUNNING
enquanto tempo restante > 0:
    executar 1 s; avançar o relógio; admitir chegadas (sem preempção)
RUNNING → TERMINATED

```

Critérios de seleção:

- FCFS: menor instante de criação.

- SJF: menor duração. Como nada é interrompido, duração e tempo restante coincidem.
- Prioridade sem preempção: maior prioridade estática.

5.2 SRTF e prioridade com preempção

```
a cada segundo:
  admitir chegadas
  se existir pronto estritamente melhor que o processo em execução:
    RUNNING → READY (preempção); devolver à lista de prontos
  se a CPU estiver livre: selecionar o melhor pronto; READY → RUNNING
  executar 1 s; se terminou: RUNNING → TERMINATED
```

"Estritamente melhor" significa:

- SRTF: tempo restante menor que o do processo em execução.
- Prioridade com preempção: prioridade estática maior que a do processo em execução.

Em caso de empate, o processo em execução permanece.

5.3 Round-Robin

```
selecionado ← cabeça da fila; READY → RUNNING
repetir até consumir o quantum ou terminar:
  executar 1 s; avançar o relógio; admitir chegadas no final da fila
se terminou: RUNNING → TERMINATED
senão: RUNNING → READY; colocar no final da fila
```

As chegadas que ocorrem exatamente no fim do quantum entram na fila antes do processo preemptado. A sequência gerada para o caso de referência coincide com o diagrama de exemplo do enunciado.

5.4 Round-Robin com prioridade e aging

```
selecionado ← pronto de maior prioridade dinâmica
READY → RUNNING; prioridade dinâmica ← prioridade estática
repetir até consumir o quantum ou terminar:
  executar 1 s; avançar o relógio; admitir chegadas (sem preempção por prioridade)
se o quantum foi consumido por completo: aging em todos os prontos
admitir quem chegou exatamente neste instante
se terminou: RUNNING → TERMINATED
senão: RUNNING → READY (disputa a próxima seleção sem preferência)
```

Regras complementares: o processo que esgota o quantum retorna à fila e disputa a próxima seleção sem preferência; o aging é aplicado quando um quantum inteiro é consumido, inclusive se o processo termina no limite; processos que chegam exatamente nesse limite são admitidos depois do aging, pois ainda não aguardaram.

6. Desempates e aleatoriedade

Ordem de desempate aplicada por todos os algoritmos com critério de seleção:

1. Processo que já ocupa a CPU. Implementado pela preempção somente diante de candidato estritamente melhor. Nos não preemptivos, não há processo na CPU no momento da escolha.
2. Menor tempo restante. Segunda passagem sobre os empatados.
3. Escolha aleatória. `SchedulingSupport.chooseRandomly` ordena os candidatos por identificador e sorteia com um `java.util.Random`.

A fonte aleatória é injetada no construtor de cada escalonador:

- na execução normal, `new Random()`;
- nos testes, `new Random(0)`, o que torna os resultados reproduzíveis;
- na interface gráfica, semente fixa opcional para repetir demonstrações.

No Round-Robin sem prioridade, a ordem FIFO define a escolha, e o sorteio nunca é usado.

7. Métricas e trocas de contexto

`metrics.MetricsCalculator`:

Métrica	Fórmula
Turnaround de um processo	conclusão – criação
Espera de um processo	turnaround – duração
<code>tt</code> e <code>tw</code>	médias aritméticas, sem arredondamento interno
Trocas de contexto	pares de segundos consecutivos executados por processos diferentes

Não contam como troca a carga inicial, a entrada ou a saída de OCIOSO e a continuidade do mesmo processo depois de uma decisão. As médias são apresentadas com duas casas decimais somente na saída.

8. Entrada, saída e erros

- Entrada:
 - o único argumento é o caminho da configuração;
 - os processos vêm de `stdin`, três inteiros por linha, separados por espaços ou tabulações;
 - a entrada é lida em UTF-8, e um BOM inicial é aceito.
- Saída:
 - sete seções, na ordem do enunciado, separadas por uma linha vazia;
 - `##` indica o processo em execução, e `- -` indica quem não executa naquele segundo.
- Erros:
 - mensagem em `stderr`;
 - `stdout` vazio;
 - código 1 para entrada ou configuração inválida e 2 para uso incorreto.

A saída só é escrita depois de montada por completo, e nunca há resultado parcial.

9. Interface gráfica

A interface (gui) usa exatamente a mesma lista de `AlgorithmReport` da saída textual. Ela não contém regras de seleção, preempção, aging ou métricas.

- Entrada:
 - tabela editável, validada pelo mesmo `ProcessInputParser`;
 - importação de arquivos de processos e de configuração;
 - campos de quantum e aging e opção de semente fixa.
- Comparação: tabela com os sete algoritmos, com o melhor valor em negrito, e gráfico de barras de `tt` e `tw`.
- Aba por algoritmo:
 - diagrama de Gantt com uma linha por processo e uma da CPU, distinguindo não criado, pronto, executando e terminado;
 - animação segundo a segundo;
 - métricas e resultado por processo.

`TimelinePresentation` deriva o que é desenhado a partir do resultado já calculado. A animação apenas revela uma linha do tempo pronta.

10. Padrões de projeto

Os padrões foram adotados somente onde resolvem um problema concreto.

Padrão ou técnica	Onde	Motivo
Objeto de valor imutável (record)	<code>SchedulerConfiguration</code> , <code>SchedulingResult</code> , <code>SchedulingMetrics</code> , <code>AlgorithmReport</code> , <code>Segment</code>	Dados validados na construção e compartilháveis sem risco de alteração.
Máquina de estados explícita	<code>ProcessControlBlock</code> + <code>ProcessState</code>	Torna as transições do processo verificáveis e fáceis de explicar.
Injeção de dependências por construtor	<code>Random</code> nos escalonadores, <code>Supplier<Random></code> no <code>SimulationRunner</code> , fluxos na <code>SchedulerApplication</code>	Testes determinísticos e testáveis sem processo externo.
Model-View (Swing)	<code>ProcessTableModel</code> , <code>ComparisonTableModel</code> , <code>ProcessResultTableModel</code>	Separa os dados exibidos dos componentes; os modelos são testados sem janela.
Observer (listeners do Swing)	botões, abas, <code>Timer</code> da animação	Mecanismo padrão do Swing para eventos.

Padrões deliberadamente não usados:

- Strategy / interface comum de escalonadores. Uma abstração `Scheduler` com um laço genérico e políticas plugáveis reduziria linhas, mas esconderia as diferenças teóricas: seleção uma vez versus a cada segundo, preempção versus quantum, fila FIFO versus seleção por critério. O `SimulationRunner` chama os sete algoritmos explicitamente.
- Template Method. Pelo mesmo motivo. O que é realmente comum (cópia, admissão e sorteio) está em `SchedulingSupport`, como funções simples.

11. Testes

A suíte JUnit 5 (`./gradlew clean test`) tem mais de 800 testes:

- Unidade: parsers, PCB, cada algoritmo (caso de referência e CTs específicos), métricas, formatação e modelos da interface.
- Integração: saída completa do caso de referência comparada com `src/test/resources/cases/reference/expected-output.txt`, arquivo gerado a partir da tabela de oráculos e não pelo simulador; execução de todos os casos de `CASES/`; contrato de erro.
- Propriedades: centenas de entradas geradas com semente fixa, verificando:
 - as invariantes do CT-18;
 - as métricas contra um cálculo independente;
 - a otimalidade do SRTF para o turnaround médio;
 - o determinismo.
- Interface: renderização do diagrama em memória (modo headless), com verificação da cor das células.

Os resultados esperados foram definidos antes das implementações.

12. Limitações conhecidas

- O tempo é discreto, em segundos inteiros, como na entrada do enunciado.
- O custo da troca de contexto não é simulado; as trocas são apenas contadas.

- Não há operações de entrada e saída nem estado bloqueado.
- A escolha aleatória, fora dos testes, pode variar entre execuções.
- A janela, os botões e a animação da interface são verificados manualmente; a lógica de apresentação e o desenho do diagrama têm testes automatizados.